A. Supreeth Reddy
2303A51392
Batch 06
AIAC Lab 1.3

Task 1: AI-Generated Logic Without Modularization (Fibonacci Sequence Without Functions)

```python
write a python code for fibonacci series without functions
n = int(input("Enter the number of terms: "))
a, b = 0, 1
if n <= 0:
    print("Please enter a positive integer.")
elif n == 1:
    print("Fibonacci sequence up to", n, ":")
    print(a)
else:
    print("Fibonacci sequence up to", n, ":")
    for i in range(n):
        print(a, end=' ')
        a, b = b, a + b
```

```
Enter the number of terms: 5
Fibonacci sequence up to 5 :
0 1 1 2 3
```

Task 2: AI Code Optimization & Cleanup (Improving Efficiency)

```python
#optimize the fibonacci code improving performance and readability
n = int(input("Enter the number of terms: "))
if n <= 0:
    print("Please enter a positive integer.")
elif n == 1:
    print("Fibonacci sequence up to", n, ":")
    print(0)
else:
    print("Fibonacci sequence up to", n, ":")
    a, b = 0, 1
    for _ in range(n):
        print(a, end=' ')
        a, b = b, a + b
```

```
Enter the number of terms: 5
Fibonacci sequence up to 5 :
0 1 1 2 3
```

Task 3: Modular Design Using AI Assistance (Fibonacci Using Functions)

```python
#optimize the fibonacci code by simplifying logic and variable usage
#use functions to make the code more modular and reusable
def fibonacci(n):
    if n <= 0:
        return "Please enter a positive integer."
    elif n == 1:
        return [0]
    else:
        sequence = [0, 1]
        for i in range(2, n):
            next_term = sequence[-1] + sequence[-2]
            sequence.append(next_term)
        return sequence
```

```
n = int(input("Enter the number of terms: "))
print("Fibonacci sequence up to", n, ":")
print(fibonacci(n))
```

```
Enter the number of terms: 5
Fibonacci sequence up to 5 :
[0, 1, 1, 2, 3]
```

Task 4: Comparative Analysis – Procedural vs Modular Fibonacci Code

1. Code Clarity:
   Procedural: Logic is written directly in the main program, which can become cluttered as the program grows.
   Modular: Logic is separated into a function, making the code cleaner and easier to understand.

2. Reusability:
   Procedural: Cannot be easily reused in other programs.
   Modular: Function can be reused multiple times in different programs.

3. Debugging Ease:
   Procedural: Harder to debug because everything is in one block.
   Modular: Easier to debug since errors can be isolated inside the function.

4. Suitability for Larger Systems:
   Procedural: Not suitable for large-scale applications.
   Modular: More suitable for large systems due to better structure.

5. Scalability:
   Procedural: Difficult to extend and maintain.
   Modular: Easy to extend and maintain.

Conclusion:
The modular approach is preferred for professional software development because it improves clarity, reusability, and maintainability.

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches for Fibonacci Series)

```
#give fibonacci code in recursive and iterative way
#include time and space complexity for both cases
# Iterative approach
def fibonacci_iterative(n):
    if n <= 0:
        return "Please enter a positive integer."
    elif n == 1:
        return [0]
    else:
        sequence = [0, 1]
        for i in range(2, n):
            next_term = sequence[-1] + sequence[-2]
            sequence.append(next_term)
        return sequence
# Recursive approach
def fibonacci_recursive(n):
    if n <= 0:
        return "Please enter a positive integer."
    elif n == 1:
        return [0]
    elif n == 2:
        return [0, 1]
    else:
```

```python
        sequence = fibonacci_recursive(n - 1)
        next_term = sequence[-1] + sequence[-2]
        sequence.append(next_term)
        return sequence
# Time and Space Complexity
print("Iterative Approach:")
print("Time Complexity: O(n)")
print("Space Complexity: O(n) due to the list storing the sequence.\n")
print("Recursive Approach:")
print("Time Complexity: O(2^n) due to the exponential growth of function calls.")
print("Space Complexity: O(n) due to the call stack in the worst case.")
# Example usage
n = int(input("Enter the number of terms: "))
print("Fibonacci sequence (Iterative):", fibonacci_iterative(n))
print("Fibonacci sequence (Recursive):", fibonacci_recursive(n))
```

```
Iterative Approach:
Time Complexity: O(n)
Space Complexity: O(n) due to the list storing the sequence.

Recursive Approach:
Time Complexity: O(2^n) due to the exponential growth of function calls.
Space Complexity: O(n) due to the call stack in the worst case.
Enter the number of terms: 5
Fibonacci sequence (Iterative): [0, 1, 1, 2, 3]
Fibonacci sequence (Recursive): [0, 1, 1, 2, 3]
```